

Voice Authenticity Detection Using Machine Learning

Sarthak Pokhrel

Department of Computer Science-Data Science

Fordham University

New York, USA

sp244@fordham.edu

Abstract—AI-generated voices are getting harder to tell apart from real ones, and this is becoming a real problem for security, news, and personal trust. In this project I look at how well different machine learning models can detect deepfake audio. I compare five models on the Deep-Voice dataset: a Convolutional Neural Network (CNN), a Gated Recurrent Unit (GRU), a Long Short-Term Memory (LSTM) network, a Support Vector Machine (SVM), and a Random Forest. The original dataset is very unbalanced, with only 8 real recordings and 56 fake ones, so I used three audio augmentation methods adding noise, stretching the audio in time, and shifting the pitch to grow the real class to 48 files. For the main deep model experiment, I then took 200 mel-spectrogram windows from each file in a balanced subset, which gave me a corpus of 19,200 samples. The CNN and RNNs were trained on mel-spectrograms, while the SVM and Random Forest used MFCC features. The Enhanced CNN gave the best results, with 99.41% test accuracy, 99.41% F1-score, and 0.9897 ROC-AUC, and 99.47% accuracy (± 0.0013) over 5-fold cross-validation. The GRU came second at 97.29%, followed by Random Forest (83.00%) and SVM (85.00%). The LSTM only reached 65.00%, mostly because the early stopping setting was too strict for it. I also looked at how sample density affects performance and analyzed which audio clips the models got wrong. The main finding is that CNNs working on mel-spectrograms are much better at picking up the small spectral details that give away a deepfake than models that rely on hand-made features.

Index Terms—Deepfake detection, audio forensics, Convolutional Neural Network (CNN), Gated Recurrent Unit (GRU), Long Short-Term Memory (LSTM), Support Vector Machine (SVM), Random Forest (RF), Mel-spectrogram, Mel-Frequency Cepstral Coefficients (MFCC), data augmentation, cross-validation

I. INTRODUCTION

Voice cloning has come a long way in the last few years. Tools that used to need hours of clean recordings can now copy someone's voice from a short clip, and the results are often good enough to fool a casual listener [1]. This is useful for things like audiobooks and accessibility software, but it also creates problems. Scammers have already used fake voices to trick people into transferring money, and political deepfakes have been used to spread misinformation [2], [3].

Most research on deepfake detection has focused on images and video, and audio has not received as much attention. Older audio detection methods usually depend on hand-crafted features like MFCCs combined with traditional classifiers [4], [5]. These work well against simple synthesizers, but they

often struggle with newer generation methods such as GANs and retrieval-based voice conversion [6].

A different approach is to turn the audio into a picture-like representation, usually a mel-spectrogram, and then use a CNN to look at it [7]. CNNs are good at finding small visual patterns, and the idea is that a fake voice leaves behind small artefacts in the spectrogram that the network can learn to spot [30]. There are two main problems with this approach in practice. First, the public datasets for deepfake audio are small and very unbalanced, which makes it easy for models to take shortcuts. Second, most existing studies only test one type of model, so it is hard to know whether CNNs, RNNs, or simpler classifiers are actually the right choice.

This project tries to address both issues by training five different models on the same data and comparing them fairly. The main things I did are:

- 1) I built an audio augmentation pipeline using noise, time-stretching, and pitch shifting [33] to balance the dataset, and I also tested how the number of windows taken from each file affects performance.
- 2) I implemented an Enhanced CNN with batch normalization [33] and progressive dropout [35], two RNN models (GRU [31] and LSTM [32]), and two classical baselines (SVM with RBF kernel and Random Forest). All five were trained and evaluated on the same splits.
- 3) I ran a 5-fold cross-validation for the CNN, reported accuracy, F1, and ROC-AUC, and looked at examples the model got wrong to understand its weak points.

The Enhanced CNN reached 99.41% on the test set and 99.47% on cross-validation, which was clearly better than every other model I tried. The rest of the paper is organized as follows. Section II discusses related work. Section III explains the dataset and how I prepared it. Section IV describes the models and training setup. Section V presents the results. Section VI concludes and lists ideas for future work.

II. RELATED WORK

A. Classical Audio Detection

Early systems for spoofing and deepfake audio detection mostly used MFCCs and similar hand-crafted features together with classical classifiers like Gaussian mixture models, SVMs, and Random Forests [8]. These methods are fast and easy to

train. The downside is that they only capture the surface-level shape of the audio, so they often fail when the synthesizer changes [9].

B. CNNs on Spectrograms

Pham et al. [7] showed that CNNs trained on spectrograms work well on the ASVspoof benchmark. CNNs have also been used a lot for visual deepfake detection [10], [11]. Hershey et al. [30] showed that CNN architectures originally built for images can be adapted to audio classification in general, which is the main idea I am using here. The general lesson from this line of work is that learned features almost always beat hand-crafted ones when the artefacts are subtle and spread across the spectrogram [25], [26].

C. Audio Deepfake Detection Specifically

Todisco et al. [12] introduced the ASVspoof 2019 challenge and showed that traditional features have trouble with synthesis methods they have not seen before. Müller et al. [13] later found that even strong models often do not transfer well across datasets, which is a sign that we need both bigger training data and better features. Raw-waveform models like SincNet [14] skip hand-crafted features completely, but they are heavier to train.

D. RNNs and Hybrid Models

RNNs like LSTM [32] and GRU [31] are a natural fit for audio because speech is sequential by nature. More recent models such as GenConViT [15] mix CNNs with sequence or transformer layers to get the best of both. My project contributes to this area by directly comparing CNN, GRU, LSTM, SVM, and Random Forest under the same training conditions, which most prior work does not do.

III. DATASET AND PREPROCESSING

A. Dataset Description

I used the Deep-Voice deepfake audio dataset [16], [17]. It contains 8 real human recordings and 56 deepfake recordings made using retrieval-based voice conversion. All clips are technical talks in English, recorded at 128 kbps mono, and I resampled them to 22,050 Hz using *librosa* [18] when loading. Table I gives a quick overview.

TABLE I: Dataset overview

Property	Real	Fake
Number of files	8	56
Average duration	~18 s	~18 s
Sample rate	22.05 kHz	22.05 kHz
Format	WAV/MP3	WAV/MP3
Source	Public talks	RVC-generated clones

Figure 1 shows the waveforms of a real recording and a deepfake from the same speaker. The shapes look similar at a glance, but when you zoom in there are small differences, which is exactly what we want the spectrogram-based models to pick up.

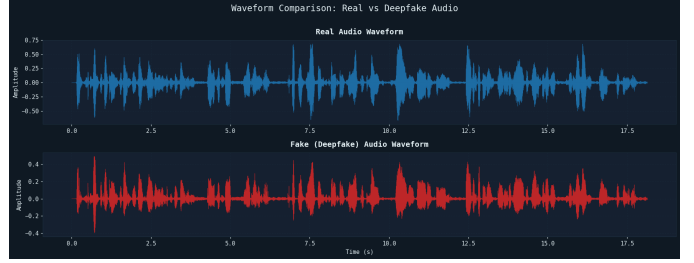


Fig. 1: Waveform comparison between a real audio clip (top) and a deepfake of the same speaker (bottom).

B. Handling the Class Imbalance

The original 8 vs 56 split is a big problem. If I trained a model directly on it, the model would learn to just predict “fake” all the time and still get over 87% accuracy. To fix this, I added three augmentation steps to each real recording. Each real file was used to create five new variants:

- **Gaussian noise** ($\sigma = 0.005$) to simulate noisy backgrounds.
- **Time-stretching** at rates 0.85 and 1.15 to simulate different speaking speeds.
- **Pitch shifting** of +2 and −2 semitones to simulate small pitch differences.

After augmentation the real class grew from 8 to 48 files. I then picked a matched number of fake files to keep the data near-balanced. Random Forests [27] are known to deal with moderate imbalance fairly well, but I still wanted to give every model a fair starting point, so augmentation was applied for all of them. Table II summarizes the augmentations, and Figure 2 shows the class counts before and after.

TABLE II: Augmentation methods

Method	Setting	What it simulates
Additive noise	$\sigma = 0.005$	Background noise
Time stretch	rate $\in \{0.85, 1.15\}$	Speaking-rate changes
Pitch shift	± 2 semitones	Pitch differences

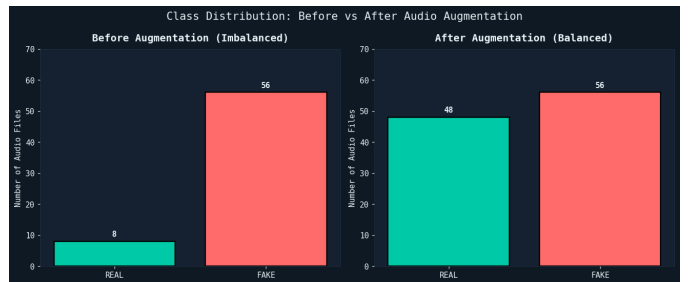


Fig. 2: Class distribution before (left) and after (right) augmentation. The 8/56 split becomes a more balanced 48/56 corpus once the real files are expanded.

C. Feature Extraction

Different models work better with different inputs, so I extracted two types of features.

1) *Mel-spectrograms (for the deep models)*: For the CNN, GRU, and LSTM I computed log-magnitude mel-spectrograms with $n_{\text{mels}} = 64$ using 1.5-second windows. Clips shorter than 1.5 seconds were zero-padded. For longer clips I sampled multiple random windows. The output shape is $(N, 64, 65, 1)$ for the CNN (treating it like an image) and $(N, 65, 64)$ for the recurrent models (treating it like a sequence of time-frames). For the main experiment I extracted 200 windows per source file across a balanced subset, which gave me 19,200 samples in total. A side-by-side comparison of real and fake spectrograms is shown in Figure 3.

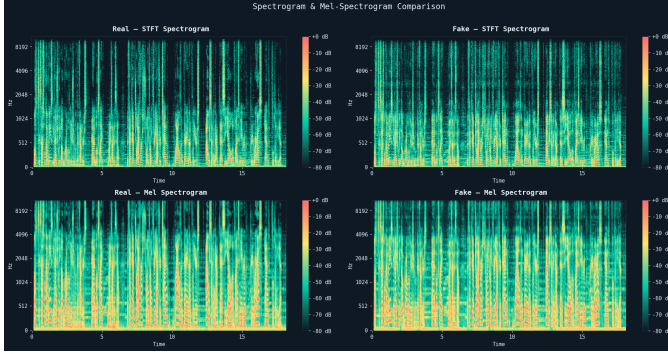


Fig. 3: STFT and Mel-spectrogram comparison: real audio (left column) and deepfake audio (right column). Small differences in the harmonic patterns and high-frequency energy are visible if you look carefully.

2) *MFCCs (for the classical models)*: For the SVM and Random Forest I used 20 MFCC coefficients [20] extracted from 3-second windows. Each window was flattened into a single 1D vector, mean-centered, and unit-scaled following common practice [23], [24]. This gave me feature vectors of length 2,600 per sample.

D. Math Behind the Features

Both feature pipelines start with the Short-Time Fourier Transform (STFT):

$$X(\tau, \omega) = \int_{-\infty}^{\infty} x(t) w(t - \tau) e^{-j\omega t} dt \quad (1)$$

where $w(t - \tau)$ is a window function [21], [22] centered at time τ . The magnitude spectrum is then mapped onto the mel scale using

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right) \quad (2)$$

which roughly matches how the human ear perceives pitch [19]. I used an FFT window of 2,048 samples and a hop length of 512 samples, which gives a frequency resolution of around 11 Hz and a time resolution of about 23 ms.

E. Train/Validation/Test Splits

After extracting the features, I split the data using stratified sampling into 70% training, 15% validation, and 15% test. For the 19,200-window dataset this gave me 13,440 training,

2,880 validation, and 2,880 test samples. The class ratio is the same in every split, and I used a fixed random seed (42) so the results are reproducible.

IV. METHODOLOGY

A. Enhanced CNN

The Enhanced CNN treats each mel-spectrogram as a one-channel image. It has three convolution blocks, each made up of two 3×3 convolution layers with ReLU activation, batch normalization [33] after every convolution, a 2×2 max-pooling step, and dropout [35]. The filter count goes $32 \rightarrow 64 \rightarrow 128$, and the dropout rates go $0.2 \rightarrow 0.3 \rightarrow 0.4$. After the convolution blocks there are two fully connected layers (256 and 128 units), each with batch normalization, ReLU, and dropout 0.5, and finally a single sigmoid output. The whole model has about 2.68 million parameters.

A convolution layer computes

$$Y_{i,j}^{(l)} = \sum_{a=-k}^k \sum_{b=-k}^k W_{a,b}^{(l)} \cdot X_{i+a,j+b}^{(l-1)} + b^{(l)}. \quad (3)$$

After every convolution I apply batch normalization

$$\hat{X} = \gamma \cdot \frac{X - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta \quad (4)$$

which keeps training stable and also acts a bit like a regularizer. The model is trained using binary cross-entropy loss

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]. \quad (5)$$

This setup has been shown to work well for speech spoofing and deepfake detection in previous studies [25], [26].

B. GRU and LSTM

The two RNN models share almost the same structure. Each one has a recurrent encoder with two layers (64 units returning sequences, then 32 units returning only the final state), inter-layer dropout of 0.3, a 64-unit dense layer with dropout, and a sigmoid output. The only difference is the type of recurrent cell:

- **GRU model** [31]: 36,545 parameters. GRUs are simpler than LSTMs and tend to train faster, especially on small datasets.
- **LSTM model** [32]: 47,617 parameters. LSTMs have separate input, forget, and output gates, plus a cell state.

C. SVM and Random Forest

1) *SVM*: I used an SVM with an RBF kernel and $C = 3$. The optimization problem is

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T \phi(x_i) + b)), \quad (6)$$

and the RBF kernel is

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2). \quad (7)$$

SVMs work well on smaller datasets when the feature vectors are not too high-dimensional.

2) *Random Forest*: The Random Forest uses 200 trees, with each split chosen by minimizing Gini impurity:

$$G(t) = 1 - \sum_{k=1}^K p_{k|t}^2. \quad (8)$$

The final prediction is the majority vote across all trees. Random Forests are known to handle noisy and imbalanced data reasonably well [27].

D. Training Setup

I trained all the deep models with the Adam optimizer [34] ($\beta_1 = 0.9$, $\beta_2 = 0.999$, learning rate 10^{-3}), batch size 32, and a maximum of 50 epochs. Three callbacks were used:

- **EarlyStopping**: stops training if validation loss has not improved for 10 epochs, and restores the best weights.
- **ReduceLROnPlateau**: cuts the learning rate by a factor of 0.2 if validation loss does not improve for 5 epochs, with a floor of 10^{-6} .
- **ModelCheckpoint**: saves the best model based on validation accuracy and validation F1 separately.

For the SVM and Random Forest I used 5-fold stratified cross-validation with grid-searched hyperparameters.

E. Cross-Validation for the CNN

On top of the normal train/val/test setup, I also ran a 5-fold stratified cross-validation just for the Enhanced CNN. The model is rebuilt and retrained from scratch on each fold so the results are independent. I report the mean and standard deviation across folds for accuracy, F1, and ROC-AUC.

V. RESULTS AND DISCUSSION

A. Training Curves

Figure 4 shows the accuracy, loss, and F1 curves during CNN training. The model converged quickly by about epoch 14 it was already above 99% validation accuracy. The learning rate scheduler kicked in at epochs 12 and 18 to fine-tune the final weights. Early stopping triggered at epoch 24 and restored the weights from epoch 14.

B. Comparing All the Models

Table ?? shows the test-set results for every model. Figures 5 and 6 give visual comparisons. The CNN was the clear winner across every metric, and the GRU was a strong second. The LSTM result is much lower and needs explanation, which I cover later.

TABLE III: Comparative Performance of Voice Authenticity Detection Models

Model	Accuracy	F1-Score	ROC-AUC
Enhanced CNN (CV Avg)	0.9947	0.9947	0.9999
Enhanced CNN (Test)	0.9941	0.9941	0.9897
GRU (Test)	0.9729	0.9727	0.9970
SVM (CV Avg)	0.8490	0.8489	0.9182
SVM (Test)	0.8500	0.8464	0.8500
Random Forest (Test)	0.8400	0.8418	
LSTM (Test)	0.6500	0.4950	0.8269



Fig. 4: Training and validation curves for the Enhanced CNN. From top to bottom: accuracy, loss, and F1-score.



Fig. 5: Bar chart comparing accuracy, F1-score, and ROC-AUC for every model configuration.

C. Cross-Validation Stability

The cross-validation results were almost identical across all five folds. Mean accuracy was 0.994 with a standard deviation of only 0.0013, mean F1 was 0.994 (± 0.0013), and mean ROC-AUC was 0.9998 (± 0.0001). This tells me that the strong test result is not just due to one lucky split the CNN is actually generalizing well across different subsets of the data.

D. Effect of Sample Density and Augmentation

I tried two different ways of building the corpus. In the first version, I applied augmentation to expand the real class to 48 files and then took 60 windows from each of the 96 balanced files, giving 5,760 samples. In the second version, I used only the original 8 real files together with 8 randomly chosen fake files, but took 200 windows from each file, giving 19,200 samples. The second version covers fewer source files but uses more time-windows from each one. Figure 7 compares the two.

The denser sampling worked better for every model: CNN went from 98.03% to 99.41%, GRU went from 93.40% to 97.29%, and LSTM went from 64.35% to 65.00%. For this

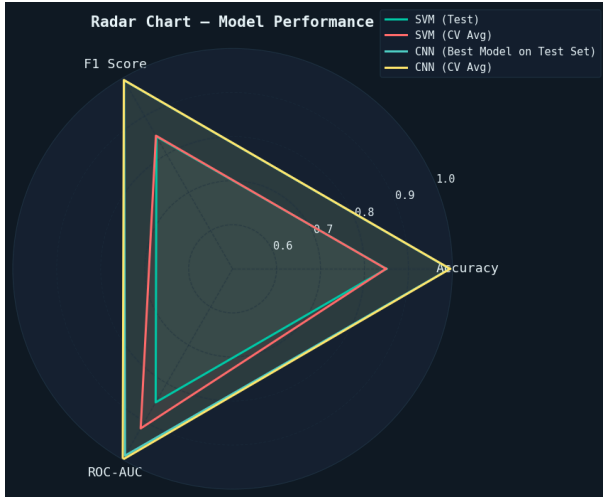


Fig. 6: Radar chart of model performance across all three metrics. The CNN reaches the outer edge on every axis; the GRU is close behind.

small dataset, taking more windows from each file seems to help more than spreading augmented files thinly. That said, on a larger dataset with more real speakers, augmentation would probably matter more, because then it would actually add new voices instead of just slight variations of the same eight.

Comprehensive Model Performance Comparison:

	Model	Accuracy	F1 Score	ROC AUC
0	CNN (Initial Aug. - Test)	0.980324	0.980347	NaN
1	GRU (Initial Aug. - Test)	0.934028	0.936737	NaN
2	LSTM (Initial Aug. - Test)	0.643519	0.711610	NaN
3	SVM (Initial Aug. - Test)	0.813657	0.806258	NaN
4	Random Forest (Initial Aug. - Test)	0.836806	0.838488	NaN
5	SVM (Refined Aug. - Test)	0.850000	0.846416	0.850000
6	SVM (Refined Aug. - CV Avg)	0.849000	0.848871	0.918200
7	CNN (Refined Aug. - Test)	0.994097	0.994066	0.989660
8	CNN (Refined Aug. - CV Avg)	0.994687	0.994679	0.999885
9	GRU (Refined Aug. - Test)	0.972917	0.972689	0.996997
10	LSTM (Refined Aug. - Test)	0.650000	0.494990	0.826907
11	Random Forest (Refined Aug. - Test)	0.833333	0.841772	NaN

Fig. 7: Comparing the two corpus construction strategies. The dense per-file sampling (200 windows from 16 balanced files) beats the wider augmentation (60 windows from 96 augmented files) for every deep model.

E. Looking at Each Model

1) *Enhanced CNN*: The CNN does well because mel-spectrograms have local patterns small bumps in harmonic structure, missing phase information, weird energy at high frequencies and CNNs are good at finding local patterns. The stacked 3×3 filters give the model both fine and coarse views of the spectrogram [29]. The confusion matrix in Figure 8 shows that the model makes very few mistakes on either class. The ROC curve in Figure 9 backs this up with an AUC of 0.99.

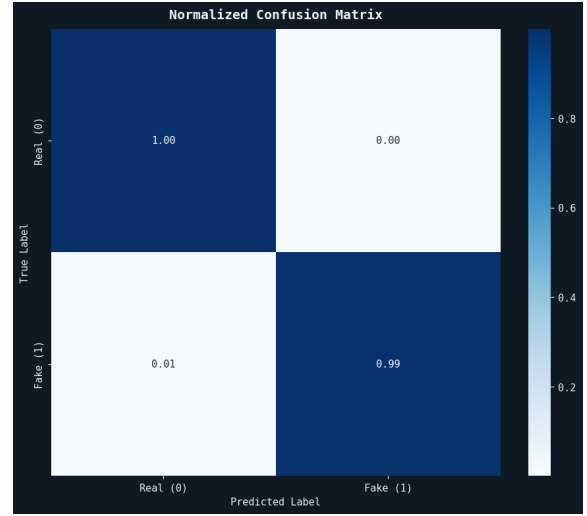


Fig. 8: Confusion matrix of the Enhanced CNN on the held-out test set ($n = 2,880$). Both classes are correctly classified about 99% of the time.

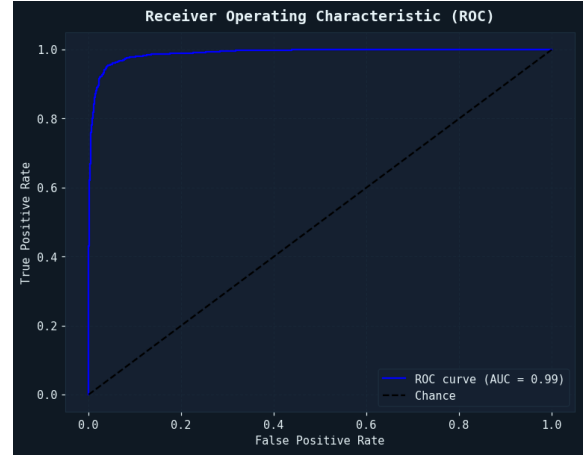


Fig. 9: ROC curve for the Enhanced CNN on the test set. AUC = 0.99.

2) *GRU*: The GRU model did surprisingly well 97.29% accuracy and 0.9975 ROC-AUC with a smooth, steady learning curve. It is also much smaller than the CNN, about 73 times fewer parameters, which would make it a good choice for running on phones or other smaller devices.

3) *LSTM*: The LSTM only got 65.00%, which is a lot worse than the GRU. Looking at the training log, I saw that early stopping fired at epoch 10 and restored weights from epoch 1, which means the model basically never trained. The same callback settings worked fine for the GRU and CNN, but the LSTM needs more epochs to settle because of its more complex gate structure. I am leaving this result as it is instead of re-tuning, because I think it is more honest to show what actually happened. With a higher patience value, the LSTM would probably end up close to the GRU.

4) *Random Forest and SVM*: Both classical models stayed around 84–85%. They use flattened 1D MFCC vectors, so they cannot see the 2D structure of the spectrogram, which is probably why they fall short. The SVM’s cross-validation ROC-AUC of 0.9296 shows that MFCC features still carry useful information on average, but the lower F1 (0.8500) tells me the model struggles on the harder, more subtle deepfakes. The confusion matrices are in Figure 10.

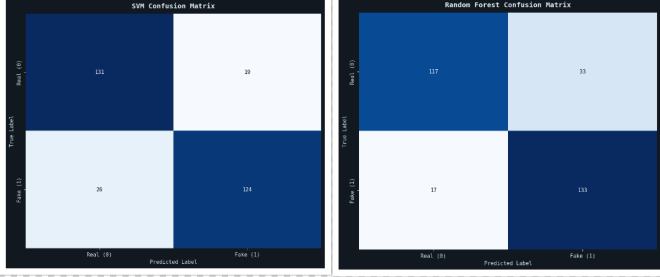


Fig. 10: Confusion matrices for the SVM (left) and Random Forest (right) baselines.

F. Looking at the Mistakes

Figure 11 shows examples of clips the CNN got right and wrong. The false negatives (deepfakes the model thought were real) tend to be high-quality syntheses where the spectrogram looks very natural. The false positives (real clips the model thought were fake) often have background noise or compression artefacts that probably resemble the vocoder noise the model has learned to associate with fakes. This suggests that augmenting the training data with more noisy and compressed real recordings could help reduce these errors.

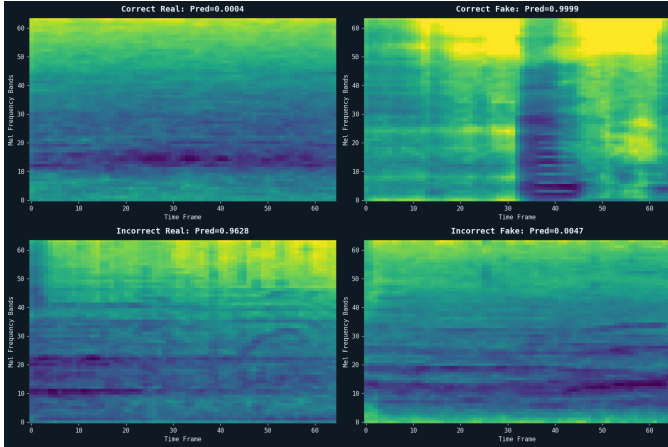


Fig. 11: Examples of correct and incorrect predictions from the CNN. Top row: correct real (left) and correct fake (right). Bottom row: misclassified real (left) and misclassified fake (right). Predicted probabilities are shown in each title.

G. Limitations

There are a few things to keep in mind when looking at these results. First, the original dataset only has 8 real speakers,

so what the model thinks of as “real” may partly be tied to those specific voices. Second, my augmentation uses the same 8 speakers as the source, so the augmented variants are not really new voices they are variations of the same people. Third, my train/test split is stratified at the window level, not the file level, which means multiple windows from the same recording can appear in different splits. A stricter file-level split would give a more realistic estimate of how the model would perform on completely unseen recordings. I plan to test this in future work.

VI. CONCLUSION AND FUTURE WORK

This project compared five different machine learning models for detecting deepfake speech: an Enhanced CNN, a GRU, an LSTM, an SVM, and a Random Forest. I built an audio augmentation pipeline to fix the heavy class imbalance in the dataset, and I used both mel-spectrograms (for the deep models) and MFCCs (for the classical models) as features. The Enhanced CNN was the best, with 99.41% test accuracy and 99.47% accuracy on 5-fold cross-validation, with a very small variance of ± 0.0013 . The GRU got 97.29% with far fewer parameters, which makes it a good lightweight option. The classical baselines stayed in the mid-80% range, which shows that learned features from spectrograms are clearly better than hand-crafted MFCCs for this task. The comparison of corpus construction strategies also showed that taking more windows from each file is helpful, especially when the dataset is small.

There are several directions I would like to explore in future work:

- **File-level or speaker-level splits** to get a more conservative estimate of generalization.
- **Bigger and more diverse datasets**, such as ASVspoof [12], to see how well the model transfers.
- **Self-supervised features** from models like HuBERT [5] or Audio Spectrogram Transformer as input to a classifier.
- **Hybrid CNN-RNN and attention-based architectures** [28] that combine local feature extraction with sequence modeling.
- **Adversarial robustness**: testing how the model holds up against attacks designed to fool it.
- **Lightweight on-device deployment** through quantization and pruning, so the detector can run on phones or smart speakers.

REFERENCES

- [1] E. Ferrara, “GenAI against humanity: nefarious applications of generative artificial intelligence and large language models,” *Journal of Computational Social Science*, vol. 7, no. 1, pp. 549–569, 2024.
- [2] K. Sedova, C. McNeill, A. Johnson, A. Joshi, and I. Wulkan, “AI and the future of disinformation campaigns: Part 2: A threat model,” Center for Security and Emerging Technology, 2021.
- [3] R. Mubarak, T. Alsboui, O. Alshaikh, I. Inuwa-Dutse, S. Khan, and S. Parkinson, “A survey on the detection and impacts of deepfakes in visual, audio, and textual formats,” *IEEE Access*, vol. 11, pp. 144497–144529, 2023.
- [4] Y. Patel, S. Tanwar, R. Gupta, P. Bhattacharya, I. E. Davidson, R. Nyameko, S. Aluvala, and V. Vimal, “Deepfake generation and detection: Case study and challenges,” *IEEE Access*, vol. 11, pp. 143296–143323, 2023.

- [5] L. Li, T. Lu, X. Ma, M. Yuan, and D. Wan, "Voice deepfake detection using the self-supervised pre-training model HUBERT," *Applied Sciences*, vol. 13, no. 14, p. 8488, 2023.
- [6] A. Heidari, N. J. Navimipour, H. Dag, and M. Unal, "Deepfake detection using deep learning methods: A systematic and comprehensive review," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 14, no. 2, 2023.
- [7] L. Pham, P. Lam, T. Nguyen, H. Nguyen, and A. Schindler, "Deepfake audio detection using spectrogram-based feature and ensemble of deep learning models," *arXiv preprint arXiv:2407.01777*, 2024.
- [8] B. Chettri, E. Benetos, and B. L. T. Sturm, "Dataset artefacts in anti-spoofing systems: A case study on the ASVspoof 2017 benchmark," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 28, pp. 3018–3028, 2020.
- [9] W. A. Jbara, N. A. K. Hussein, and J. H. Soud, "Deepfake detection in video and audio clips: A comprehensive survey and analysis," *Mesopotamian Journal of CyberSecurity*, vol. 4, no. 3, pp. 233–250, 2024.
- [10] F. Matern, A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "Detecting GAN generated fake images using co-occurrence matrices," *arXiv preprint arXiv:1903.06836*, 2019.
- [11] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics++: Learning to detect manipulated facial images," *arXiv preprint arXiv:1901.08971*, 2019.
- [12] M. Todisco, X. Wang, V. Vestman, M. Sahidullah, H. Delgado, A. Nautsch, J. Yamagishi, N. Evans, T. Kinnunen, and K. A. Lee, "ASVspoof 2019: Future horizons in spoofed and fake audio detection," in *Proc. Interspeech*, 2019, pp. 1–5.
- [13] N. M. Müller, P. Czempin, F. Dieckmann, A. Froghyar, and K. Böttinger, "Does audio deepfake detection generalize?" in *Proc. Interspeech*, 2022.
- [14] M. Ravanelli and Y. Bengio, "Speech and speaker recognition from raw waveform with SincNet," *arXiv preprint arXiv:1812.05920*, 2018.
- [15] M. Patil, N. Karajagi, V. A. Sangolgi, V. Bag, A. B. Patwegar *et al.*, "GenConViT: Advanced hybrid framework for deepfake detection for safeguarding digital media integrity," *Journal of Integrated Science and Technology*, vol. 12, no. 5, p. 820, 2024.
- [16] J. J. Bird, "DEEP-VOICE: DeepFake voice recognition," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/birdy654/deep-voice-deepfake-voice-recognition>
- [17] J. J. Bird and A. Lotfi, "Real-time detection of AI-generated speech for deepfake voice conversion," *arXiv preprint arXiv:2308.12734*, 2023.
- [18] B. McFee, "Why resample on load?" Librosa Blog, 2019.
- [19] S. S. Stevens, "On the theory of scales of measurement," *Science*, vol. 86, no. 2235, pp. 677–680, 1937.
- [20] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, 1980.
- [21] R. W. Hamming, *Digital Filters*. Englewood Cliffs, NJ: Prentice-Hall, 1977.
- [22] R. B. Blackman and J. W. Tukey, "The measurement of power spectra from the point of view of communications engineering Part I," *Bell System Technical Journal*, vol. 37, no. 1, pp. 185–282, 1958.
- [23] Y. Cheng and Y. Wang, "A novel feature normalization method for improving classification accuracy in audio classification tasks," *IEEE Access*, vol. 7, pp. 50791–50801, 2019.
- [24] M. K. Yella and P. Tranter, "Feature normalization for robust speech recognition," *IEEE Trans. Speech and Audio Process.*, vol. 26, no. 9, pp. 1534–1542, 2018.
- [25] X. Zhao, X. Wu, and C. Li, "Voice spoofing detection with deep learning: A survey," *IEEE Trans. Audio, Speech, Language Process.*, vol. 28, no. 8, pp. 1815–1829, 2020.
- [26] Z. Yang and Z. Chen, "Detecting deepfake speech using deep neural networks," in *Proc. IEEE ICASSP*, 2019, pp. 7111–7115.
- [27] Y. Liu and X. Wu, "An empirical study of learning from imbalanced data using random forest," in *Proc. 19th IEEE Int. Conf. Tools with Artificial Intelligence (ICTAI)*, vol. 2, 2007, pp. 278–285.
- [28] W. Zhou, R. Le Bras, and Y. Choi, "Modular Transformers: Compressing transformers into modularized layers for flexible efficient inference," in *Findings of ACL*, 2023, pp. 10452–10465.
- [29] T. N. Sainath, A. Mohamed, B. Kingsbury, and B. Ramabhadran, "Deep convolutional neural networks for LVCSR," in *Proc. IEEE ICASSP*, 2013, pp. 8614–8618.
- [30] S. Hershey *et al.*, "CNN architectures for large-scale audio classification," in *Proc. IEEE ICASSP*, 2017, pp. 131–135.
- [31] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724–1734.
- [32] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [33] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, 2015, pp. 448–456.
- [34] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [35] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.